

UNITÉ 6 · SEMAINES 8–9

Signaux et mémoire partagée

📅 Semaines 8–9 · dates à confirmer </> Laboratoire TD5 (corrigé + défi IA) ~ 3 animations

MANUEL M6 · THÉORIE DE L'UNITÉ

Signaux et mémoire partagée

Lecture de base de l'unité 6 (≈ 1 h 30). Les sorties sont réelles: programmes du TD5 exécutés sur la machine du cours.

SOMMAIRE

- | | | | |
|----|--|----|--|
| §1 | Signaux: envoi, attente et délivrance | §3 | <code>mmap</code> : projeter des fichiers en mémoire |
| §2 | Gestionnaires et synchronisation sans course | §4 | La mémoire partagée et son prix |

§1 Signaux: envoi, attente et délivrance

Un **signal** est une notification asynchrone adressée à un processus. Il est envoyé par le noyau (`SIGSEGV` pour un accès invalide, `SIGPIPE` pour une écriture dans un tube sans lecteur, `SIGCHLD` quand un enfant se termine), par le terminal (`Ctrl-C` → `SIGINT`, `Ctrl-\` → `SIGQUIT`, `Ctrl-Z` → `SIGTSTP`) ou par un autre processus avec `kill(pid, sig)`. Pour chaque signal, le processus a une **action** : celle par défaut (terminer, terminer avec `core`, arrêter, continuer ou ignorer), l'ignorer (`SIG_IGN`) ou exécuter une fonction **gestionnaire**. `SIGKILL` et `SIGSTOP` sont les seuls qui ne peuvent être ni capturés, ni ignorés, ni bloqués.

`kill` ne fait que marquer le signal comme **en attente**. Le noyau le **délivre** quand le processus revient en mode utilisateur — à la fin d'un appel système ou quand il obtient le CPU après un changement de contexte —, sauf s'il est **bloqué** par le masque du processus: il attend alors. Pour les signaux standard, «en attente» est un bit: si trois signaux identiques arrivent pendant qu'il est bloqué, un seul est délivré au déblocage. Les signaux temps réel (`SIGRTMIN` ...) sont bien mis en file et peuvent porter une valeur.

Pratiquez → A6.1 délivrance des signaux

§2 Gestionnaires et synchronisation sans course

Les gestionnaires s'installent avec `sigaction` (`signal()` a une sémantique différente selon les systèmes). Pendant que le gestionnaire s'exécute, le même signal reste bloqué (ainsi que ceux de `sa_mask`); avec `SA_RESTART`, un appel système interrompu reprend tout seul, sinon il échoue avec `EINTR` (unité 2). Un gestionnaire interrompt le programme à n'importe quel endroit: il ne

peut donc utiliser que des fonctions *async-signal-safe* (`write` oui; `printf` et `malloc` non) et communiquer avec le programme par des variables `volatile sig_atomic_t`.

Attendre un signal avec `pause()` comporte une **course** : si le signal arrive entre la dernière vérification et l'appel à `pause`, le gestionnaire s'exécute trop tôt et `pause` attend pour toujours. La solution consiste à **bloquer** le signal avec `sigprocmask`, vérifier la condition et attendre avec `sigsuspend(mask)`, qui remplace le masque et dort **en une seule opération atomique** : le signal arrivé «trop tôt» est resté en attente et il est délivré à l'intérieur de `sigsuspend`.

C tds/td5/mysignal.c

```
/*
 * TD5 - exercise 1: catch SIGUSR1 with sigaction(2).
 * ./mysignal [n]      handles SIGUSR1 n times (default 3), then restores the default
 * action
 * From another terminal: kill -USR1 <pid>, kill -STOP <pid>, kill -CONT <pid>, kill
 * -KILL <pid>
 * After the n-th SIGUSR1 the default action is back: the next SIGUSR1 terminates the
 * process.
 */
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

static volatile sig_atomic_t received = 0;

static void on_usr1(int sig)
{
    /* Only async-signal-safe functions here: no printf (it may be interrupted mid-
    buffer). */
    static const char msg[] = "handler: received signal ";
    char digits[4] = { (char)('0' + sig / 10), (char)('0' + sig % 10), '\n', 0 };
    if (write_all(STDERR_FILENO, msg, sizeof msg - 1) == -1 || write_all(STDERR_FILENO,
    digits, 3) == -1)
        return;
    received++;
}

int main(int argc, char *argv[])
{
    int max = argc > 1 ? atoi(argv[1]) : 3;
    struct sigaction sa = { 0 }, old;
    sigset_t block, wait_mask;
```

```
printf("pid %d: waiting for SIGUSR1 (%d times)\n", getpid(), max);
fflush(stdout);

sa.sa_handler = on_usr1;
sigemptyset(&sa.sa_mask);           /* nothing extra blocked while the handler runs
*/
sa.sa_flags = 0;
exit_if(sigaction(SIGUSR1, &sa, &old) == -1, "sigaction");

/* Block SIGUSR1 outside sigsuspend: a signal arriving between the test of
`received`
and the wait stays pending instead of being lost (the pause() race). */
sigemptyset(&bblock);
sigaddset(&bblock, SIGUSR1);
exit_if(sigprocmask(SIG_BLOCK, &bblock, &wait_mask) == -1, "sigprocmask");
sigdelset(&wait_mask, SIGUSR1);

while (received < max)
    sigsuspend(&wait_mask);          /* atomically: unblock SIGUSR1 and sleep */

exit_if(sigaction(SIGUSR1, &old, NULL) == -1, "sigaction restore");
printf("default action restored: the next SIGUSR1 terminates me\n");
fflush(stdout);
exit_if(sigprocmask(SIG_UNBLOCK, &bblock, NULL) == -1, "sigprocmask");
for (;;)
    pause();
}
```

Sortie réelle

```
$ ./mysignal 2 & P=$!; sleep 0.3; kill -USR1 $P; sleep 0.2; kill -STOP $P; kill -USR1
$P; sleep 0.2; kill -CONT $P; sleep 0.2; kill -USR1 $P; wait $P; echo "exit status: $?"
(128 + 10 = killed by SIGUSR1)"
pid 357338: waiting for SIGUSR1 (2 times)
handler: received signal 10
handler: received signal 10
default action restored: the next SIGUSR1 terminates me
bash: line 1: 357338 User defined signal 1      ./mysignal 2
exit status: 138 (128 + 10 = killed by SIGUSR1)
```

Autres outils: `alarm(s)` programme un `SIGALRM`; le `sleep` de glibc utilise `clock_nanosleep` (et non `alarm`); `signalfd` transforme les signaux en données lues depuis un descripteur, et `pidfd_send_signal` évite d'envoyer un signal à un PID déjà réutilisé.

Pratiquez → TD5 · 1 et 2

§3 mmap : projeter des fichiers en mémoire

`mmap(addr, len, prot, flags, fd, offset)` réserve une plage d'adresses et la lie à un fichier (ou à rien, avec `MAP_ANONYMOUS`). Il ne copie rien: chaque page est chargée au premier accès par un défaut de page (unité 4). Avec `MAP_SHARED`, les écritures vont dans le cache de pages du fichier, les autres processus qui le projettent les voient et elles atteignent le disque (`msync` ou à la suppression de la projection); avec `MAP_PRIVATE`, la première écriture copie la page et le fichier ne change pas. `len` est arrondi au nombre de pages supérieur; `offset` doit être un multiple de la taille de page.

Accéder au-delà de la fin du fichier dans la dernière page renvoie des zéros qui ne sont pas enregistrés; toucher une page entière sans fichier derrière déclenche **SIGBUS** (l'adresse est valide, mais il n'y a pas de données à charger). Pour agrandir un fichier projeté, appelez d'abord `ftruncate`. Les protections `PROT_*` s'appliquent page par page; sur x86, écrire implique de pouvoir lire:

Sortie réelle

```
$ ./mmap_limits
file size 128 bytes, page size 4096 bytes, mapping of 8192 bytes
wrote bytes 0..4095; byte 4096 raised SIGBUS
file size after the writes: 128 bytes
```

Sortie réelle

```
$ ./map_write
PROT = PROT_WRITE
pointer: 0x7c7095214000
read:    *p = 0
write:   *p = 42
```

Sortie réelle

```
$ ./map_exec
PROT = PROT_EXEC
pointer: 0x75b3a49d8000
bash: line 1: 357495 Segmentation fault      (core dumped) ./map_exec
[exit status 139]
```

Pratiquez → A6.3 les limites de mmap · TD5 · 3 et 4

§4 La mémoire partagée et son prix

Partager de la mémoire, c'est faire pointer des pages virtuelles de deux processus vers le même cadre physique. Il faut un **point de rendez-vous** et une interface: `mmap` d'un fichier commun avec `MAP_SHARED` ; `shm_open` POSIX + `mmap` (un nom dans `/dev/shm`, recommandé aujourd'hui) ; ou System V: `ftok(file, id)` génère une clé, `shmget` crée ou trouve le segment, `shmat` l'attache, `shmdt` le détache et `shmctl(IPC_RMID)` le détruit. Le segment survit aux processus jusqu'à ce que quelqu'un le détruise:

Sortie réelle

```
$ ./shm_counter; ./shm_counter; ./shm_counter; ./shm_counter --remove
segment 65593 (key 0x4605476d): counter = 1
segment 65593 (key 0x4605476d): counter = 2
segment 65593 (key 0x4605476d): counter = 3
segment 65593 removed
```

Deux règles. Première: chaque processus voit la zone à une **adresse différente**, donc on n'y stocke pas de pointeurs (mais des décalages). Seconde: la mémoire partagée est la communication la plus rapide parce qu'elle ne passe pas par le noyau, mais pour cette même raison **rien ne la synchronise**. Si plusieurs processus font « lire, ajouter 1, écrire » sur le même compteur, des mises à jour se perdent; avec une opération atomique du processeur (`__atomic_add_fetch`, `lock xadd` sur x86), non:

Sortie réelle

```
$ for i in 1 2 3 4 5; do ./shm_race 5 | cut -d: -f1; done
19 processes registered (expected 32)
28 processes registered (expected 32)
25 processes registered (expected 32)
23 processes registered (expected 32)
27 processes registered (expected 32)
```

Sortie réelle

```
$ ./shm_fork
16 processes registered (expected 16): 357564 357566 357567 357572 357568 357571 357569
357565 357574 357573 357570 357576 357578 357575 357577 357579
```

Pratiquez → A6.2 mémoire partagée · TD5 · 5